```
$ spring run TransactionLab --propagation=REQUIRED
```

@Transactional [illegible]務管理

// [illegible] —— [illegible]

begin
[illegible]啟事務

commit
[illegible] flush

rollback
當作沒發生

proxy
幕後功臣

```
$ cat agenda.md
```

今日路線

01 / □□□

□行轉帳與三大職責

02 / □□□□□

全域攔截器的兩套合約

03 / AOP □理

替身演員如何開關事務

04 / □□□□□

自我呼叫、rollback 規則、FAQ

目標：不再問「為什麼我的 `@Transactional` 沒生效？」

PART 01

@Transactional □□□□

□□□□□□□□□□□□

銀行轉帳的四個步驟

1.
開啟事務

2. A 扣錢
UPDATE accounts...

3. B 加錢
UPDATE accounts...

4.
Commit

WHAT IF STEP 3 FAILS?

錢不能憑空消失 —— 銀行說「當作沒發生過」（Rollback），把錢還給 A。

`@Transactional` 就是那個確保「要嘛全做，要嘛全不做」的銀行經理。

三大職責（外加一個隱藏版）

職責	說明	比喻
開啟合約	[illegible] `EntityManager`	婚禮開場白
定義行為	「這是不可分割的業務」	「你願意嗎？」
觸發回滾	出錯時通知資料庫回滾	[illegible] → [illegible]
強制同步	[illegible] `flush()`	[illegible]…[illegible] → [illegible]

最重要的一點：方[illegible]正常結束 → `commit` → [illegible]制 **`flush()`**。這就是為什麼你改了 `Entity` [illegible]呼叫 `save()`[illegible]資料還是存進去了。

PART 02

命名即命運

□□□□□□□□□□□□

寫入合約

```
// 這些方法名開頭
// 自動獲▯事務保護
add*, save*, create*,
update*, delete*
```

`PROPAGATION_REQUIRED`

「必須有事務！沒有就▯一個，有就加入」

▯ RuntimeException → 回滾

唯讀合約

```
// 這些方法名開頭
// 只能讀不能寫
find*, get*, search*,
getCount*, * (其他所有)
```

`PROPAGATION_NOT_SUPPORTED`

「不需要事務，我只是來看看」

`setReadOnly(true)`▯只看不摸

命名陷阱

```java
// ✅ 有事務保護
public void saveUser() { ... }
public void updateOrder() { ... }

// ❌ 沒有事務保護（即使你加了 @Transactional）
public void processUser() { ... }  // 不符合命名規則
public void handleOrder() { ... }  // 被 * 攔截成唯讀
```

PRIORITY

就算手動打上 `@Transactional`，攔截器的優先級更高， `process*` 還是被當成唯讀處理。

方法命名決定你有沒有事務保護

CHECKPOINT_01

關於全域事務攔截器，哪一句正確？

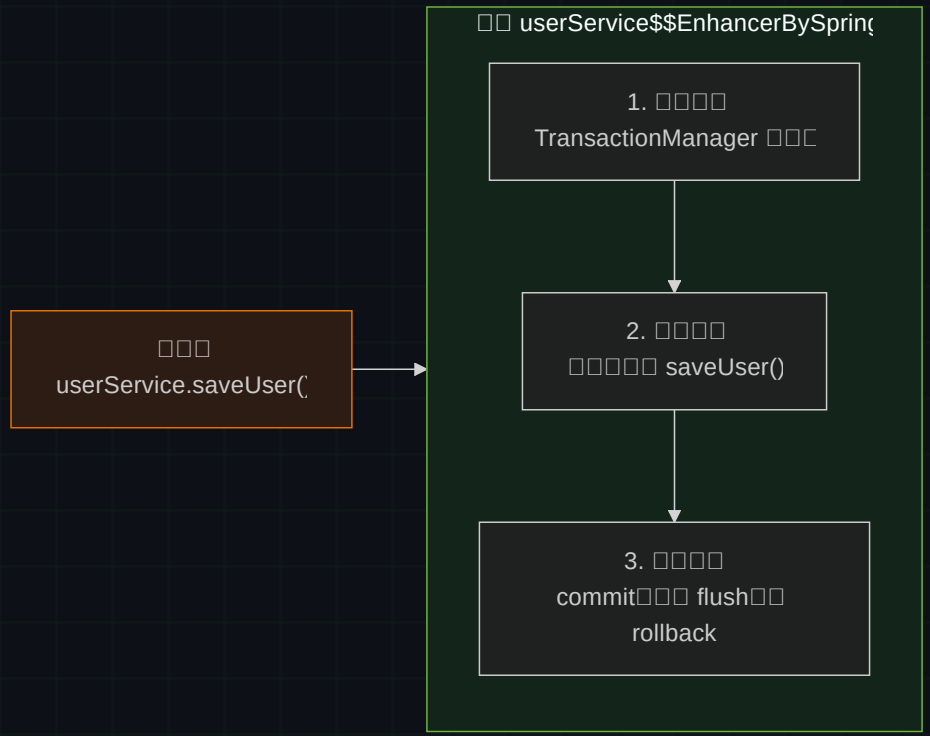
- A `processUser()` 加上 `@Transactional`，就一定有寫入事務
- B 同類別內 `this.saveUser()` 呼叫，也會經過 AOP 代理
- C 攔截器以方法名決定事務行為，`process*` 可能被當成唯讀

PART 03

AOP 代理：幕後功臣

Spring □□□□□□□□□□□□□□□

替身演員的工作流程



呼叫端

userService.saveUser(

□理 userService$$EnhancerBySpring

1. 開門營業

TransactionManager 開連□

2. 請君入甕

呼叫真正的 saveUser()

3. 結帳走人

commit（觸發 flush）或 rollback

PART 04

陷阱與最佳實踐

90% □□□□□□□□□□

自我呼叫陷阱

```java
@Service
public class UserService {

    // 🚨 沒有獨立事務!
    public void processAndSave(User user) {
        validateUser(user);
        saveUser(user); // this.saveUser()
    }

    public void saveUser(User user) {
        userRepository.save(user);
    }
}
```

WHY

`this.saveUser()` 呼叫的是□己，不是代理物件 —— AOP 完全沒機會插手。

解法 1：把方法拆到不同 Service

解法 2：注入自己，用 self.saveUser()

「為什麼沒生效」四大原因

01 [illegible]

攔截器規則優先，process* 變唯讀

02 [illegible]

this. 繞過代理

03 [illegible] public

private 是內心戲，代理看不到

04 Checked Exception

[illegible] RuntimeException

$ 下次事務沒生效，照這四條依序排查，命中率 90%。

最佳實踐

DO

- ✓ 遵守命名規則：`save* / update* / delete*`
- ✓ 事務方法保持簡短，鎖越短越好
- ✓ 只在 `Service` 層使用

DON'T

- × 事務方法太長，其他請求排隊
- × 在迴圈裡開事務，效能爆炸
- × 混用不同資料源（那是分散式事務的故事）

今日重點

生命週期管理者

開啟、監控、commit□□□ flush□□ rollback

命名決定命運

攔截器依方法名分配寫入／唯讀合約

□理是幕後功臣

this. □□□ private 方法都繞過它

唯讀優化

□□□ find* / get*□□□□ dirty checking

要嘛全做，要嘛全不做 —— 而且方法名說了算。

Transaction committed.

□□□□JPA □□□□□□□□ flush □ dirty checking □□

```
$ spring run TransactionLab --status
begin → invoke → commit(flush) | rollback
```